SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Fundamentals Of Data Science **COURSE CODE: DSA04**

COURSE OUTCOME COVERED

CO3: *Evaluate datasets using descriptive statistics, including summarization, outlier detection, and distribution analysis, and perform statistical inference through estimation and hypothesis testing. (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 02 **Total Marks:100**

Annexure A

Data Interpretation

Code of Conduct:

I M.Gayathri(192472074)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:M.Gayuathri

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary) 5 Marks	Level 3 (Proficient) 3 /4 Marks	Level 2 (Developing) 2 Marks	Level 1 (Beginning) 1 Mark	Marks
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data	
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided	
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data	
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions	
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand	

Annexure A

Problem 01: Hospital Patient Health Monitoring

Scenario

A hospital wants to investigate how patient lifestyle factors affect blood pressure levels.

Dataset

Patient_ID	Daily_Exercise (Minutes)	BMI	Systolic_BP
P001	10	32	150
P002	15	31	148
P003	20	30	145
P004	25	29	142
P005	30	28	138
P006	35	27	134
P007	40	26	130
P008	45	25	126
P009	50	24	122
P010	55	23	118

Tasks

1. Find the mean and standard deviation of blood pressure.
2. Calculate covariance between:
    - Exercise and Blood Pressure
    - BMI and Blood Pressure
3. Calculate correlation coefficients.
4. Identify whether exercise or BMI is more strongly associated with blood pressure.
5. Discuss possible healthcare interventions.

Problem 02: E-Commerce Customer Behaviour Analysis

Scenario

An e-commerce company wants to identify factors influencing customer spending.

Dataset

Customer_ID	Website_Visits	Time_Spent (Minutes)	Purchase_Amount (₹)
C001	5	20	1200
C002	8	25	1800
C003	10	30	2500
C004	12	35	3200
C005	15	40	4000
C006	18	45	5000
C007	20	50	6200

C008	23	55	7300
C009	25	60	8500
C010	28	70	10000

Tasks

1. Calculate mean, variance, and standard deviation of Purchase Amount.
2. Find covariance between:
    - Website Visits and Purchase Amount
    - Time Spent and Purchase Amount
3. Calculate correlation coefficients.
4. Determine which customer behavior metric better predicts spending.
5. Suggest marketing strategies based on the findings.

Problem 03: Employee Productivity and Performance

Scenario

A software company is investigating the relationship between employee productivity and performance ratings.

Dataset

Employee_ID	Training_Hours	Projects_Completed	Performance_Rating
E001	5	2	60
E002	8	3	65
E003	10	4	70
E004	12	5	74
E005	15	6	79
E006	18	7	84
E007	20	8	88
E008	22	9	91
E009	25	10	95
E010	28	11	98

Tasks

1. Compute the mean, variance, and standard deviation of Performance Ratings.
2. Calculate covariance between:
    - Training Hours and Performance Rating
    - Projects Completed and Performance Rating
3. Compute correlation coefficients.
4. Identify the strongest factor affecting employee performance.
5. Recommend HR policies to improve employee productivity.

Problem 04 : Smart City Traffic Analysis

A city administration wants to analyze traffic congestion patterns.

Day	Traffic_Volume (Vehicles)	Rainfall (mm)	Average_Speed (km/h)	Accident_Count
D1	12000	0	55	2
D2	13500	5	50	3
D3	14500	10	48	4
D4	16000	15	45	5
D5	17500	20	42	6
D6	19000	25	38	8
D7	21000	30	35	10
D8	22500	35	32	12
D9	24000	40	28	14
D10	26000	45	25	16

Tasks

1. Calculate descriptive statistics (Mean, Variance, Standard Deviation) for all variables.
2. Compute covariance matrices.
3. Compute correlation matrices.
4. Identify positive and negative relationships among variables.
5. Determine which factor is most associated with accident occurrence.
6. Provide data-driven recommendations to reduce traffic accidents.
7. Visualize relationships using scatter plots and interpret the results.

Problem 5: Average Delivery Time

A food delivery company records the delivery times (minutes) for 15 orders:
22, 25, 18, 30, 27, 24, 21, 19, 26, 23, 28, 20, 24, 25, 22
Tasks:

1. Find the point estimate of the population mean delivery time.
2. Find the point estimate of the population variance.
3. Interpret the estimates.

Problem 6: Customer Spending

A sample of 12 customers spent the following amounts ($):
120, 145, 130, 155, 160, 125, 140, 135, 150, 170, 165, 155
Tasks:

1. Estimate the average spending.
2. Estimate the standard deviation.
3. Explain why these are point estimates.

Problem 7: Product Ratings

A company collects ratings from 20 customers:
4, 5, 3, 4, 5, 4, 3, 5, 4, 4, 5, 3, 4, 5, 4, 3, 5, 4, 5, 4
Tasks:

1. Compute the sample mean.

2. Compute the sample variance.
3. Compute the standard error of the mean.
4. Explain what the standard error indicates.

Problem 8: Exam Scores

Scores of 25 students:
72, 75, 68, 80, 77, 73, 79, 81, 76, 74, 78, 69, 70, 82, 71, 75, 77, 73, 80, 76, 74, 79, 72, 81, 78
Tasks:

1. Calculate mean and standard deviation.
2. Calculate standard error.
3. Compare standard deviation and standard error.

Problem 9: Manufacturing Weights

A sample of 50 packets has:

- Mean weight = 498 g
- Standard deviation = 12 g

Tasks:

1. Construct a 95% confidence interval for the population mean weight.
2. Interpret the interval.
3. Determine whether the target weight of 500 g is plausible.

Problem 10: Customer Satisfaction

Two branches report:

Branch	Mean Rating	95% CI
A	4.3	(4.1, 4.5)
B	4.0	(3.8, 4.2)

Tasks:

1. Compare the confidence intervals.
2. Can you conclude Branch A performs better?
3. Explain your reasoning.

Problem 11: Average Battery Life

A battery manufacturer claims the average battery life is 10 hours.
A sample of 36 batteries gives:

- Mean = 9.5 hours
- Standard deviation = 1.8 hours

Tasks:

1. Formulate null and alternative hypotheses.
2. Conduct a hypothesis test at $\alpha = 0.05$.
3. State your conclusion.

Problem 12: Marketing Campaign

A company believes a new campaign increases sales.
Sales before campaign:

50, 52, 49, 55, 53, 51, 54, 50
Sales after campaign:
58, 60, 57, 62, 59, 61, 63, 58
Tasks:

1. Conduct a hypothesis test.
2. Calculate the p-value.
3. Interpret the p-value.
4. Make a decision at $\alpha = 0.05$.

Problem 13: Call Centre Response Time

A sample of 100 calls has:

- Mean response time = 4.8 minutes
- Standard deviation = 1.5 minutes

Tasks:

1. Estimate the population mean.
2. Construct a 95% confidence interval.
3. Explain the Frequentist interpretation.

Problem 14: Defective Products

A sample of 500 products contains 22 defective items.
Tasks:

1. Estimate the defect rate.
2. Construct a 95% confidence interval.
3. Explain the Frequentist meaning of the interval.

Problem 15: Email Spam Detection

Historical information:

- Probability an email is spam = 0.25
- Probability spam email contains keyword "Offer" = 0.80
- Probability non-spam email contains keyword "Offer" = 0.10

Tasks:

1. Compute the probability that an email is spam given it contains "Offer."
2. Apply Bayes' theorem.
3. Interpret the result.

Problem 16: Disease Diagnosis

Given:

- Disease prevalence = 2%
- Test sensitivity = 95%

- Test specificity = 90%

Tasks:

1. Compute the posterior probability of disease after a positive test.
2. Explain why the result differs from 95%.

Problem 17: Website Conversion Rate

A company tests two landing pages.

Group	Visitors	Purchases
A	5000	250
B	5200	312

Tasks:

1. Calculate conversion rates.
2. Formulate hypotheses.
3. Conduct a two-proportion z-test.
4. Determine whether page B performs better.

Problem 18: Mobile App Feature Test

Feature A:

- 400 users
- Average engagement = 18 min
- SD = 5 min

Feature B:

- 420 users
- Average engagement = 20 min
- SD = 6 min

Tasks:

1. Test whether Feature B increases engagement.
2. Calculate test statistic and p-value.
3. Interpret results.

Problem 19: Recommendation System

A new recommendation algorithm increases click-through rate from 4.8% to 5.3%.
Sample size:

- Old system: 100,000 users
- New system: 100,000 users

Tasks:

1. Test statistical significance.
2. Calculate p-value.
3. Discuss practical significance versus statistical significance.

Problem 20: Salary Prediction Model

Two machine learning models produce Mean Absolute Errors (MAE):

Model	MAE
Model A	4200
Model B	4000

Across 50 test folds, the difference in MAE has:

- Mean difference = 200
- Standard deviation = 350

Tasks:

1. Perform a paired t-test.
2. Determine whether Model B significantly outperforms Model A.
3. Explain the result in terms of statistical significance.

ANSWERS:

1.

CODE:

```
import numpy as np

# Patient data from the assessment
exercise = np.array([10, 15, 20, 25, 30,
                     35, 40, 45, 50, 55])

bmi = np.array([32, 31, 30, 29, 28,
                27, 26, 25, 24, 23])

bp = np.array([150, 148, 145, 142, 138,
               134, 130, 126, 122, 118])

# 1. Mean and standard deviation
print("HOSPITAL PATIENT HEALTH MONITORING")
print("----------------------------------")

print("Mean Blood Pressure:", round(np.mean(bp), 2))
print("Standard Deviation:", round(np.std(bp, ddof=1), 2))

# 2. Covariance
print("\nCovariance:")
print("Exercise and BP:", round(np.cov(exercise, bp)[0, 1], 2))
print("BMI and BP:", round(np.cov(bmi, bp)[0, 1], 2))

# 3. Correlation coefficients
r1 = np.corrcoef(exercise, bp)[0, 1]
r2 = np.corrcoef(bmi, bp)[0, 1]

print("\nCorrelation Coefficients:")
print("Exercise and BP:", round(r1, 4))
print("BMI and BP:", round(r2, 4))
```

```
# 4. Strongest association
print("\nAnalysis:")
if abs(r1) > abs(r2):
    print("Exercise has the stronger association with BP.")
else:
    print("BMI has the stronger association with BP.")

# 5. Healthcare interventions
print("\nHealthcare Interventions:")
print("1. Encourage regular physical exercise.")
print("2. Maintain a healthy BMI.")
print("3. Monitor blood pressure regularly.")
print("4. Follow a balanced, low-sodium diet.")
```

OUTPUT:

```
= RESTART: C:/Users/Madaraboina Gayathri/OneDrive/Desktop/DSA0402-FODS/ASSESSMENTS/CO3-AT2/Q1.py
HOSPITAL PATIENT HEALTH MONITORING
----------------------------------
Mean Blood Pressure: 135.3
Standard Deviation: 11.14

Covariance:
Exercise and BP: -168.06
BMI and BP: 33.61

Correlation Coefficients:
Exercise and BP: -0.9969
BMI and BP: 0.9969

Analysis:
Exercise has the stronger association with BP.

Healthcare Interventions:
1. Encourage regular physical exercise.
2. Maintain a healthy BMI.
3. Monitor blood pressure regularly.
4. Follow a balanced, low-sodium diet.
```

2.

CODE:

```
import numpy as np

# Customer data
visits = np.array([5, 8, 10, 12, 15, 18, 20, 23, 25, 28])
time_spent = np.array([20, 25, 30, 35, 40, 45, 50, 55, 60, 70])
purchase = np.array([1200, 1800, 2500, 3200, 4000,
                     5000, 6200, 7300, 8500, 10000])

print("E-COMMERCE CUSTOMER BEHAVIOUR ANALYSIS")
print("--------------------------------------")

# 1. Mean, variance and standard deviation
print("\n1. Purchase Statistics")
print("Mean:", round(np.mean(purchase), 2))
print("Variance:", round(np.var(purchase, ddof=1), 2))
```

```
print("Standard Deviation:", round(np.std(purchase, ddof=1), 2))

# 2. Covariance
print("\n2. Covariance")
print("Website Visits and Purchase:",
      round(np.cov(visits, purchase)[0, 1], 2))
print("Time Spent and Purchase:",
      round(np.cov(time_spent, purchase)[0, 1], 2))

# 3. Correlation coefficients
r1 = np.corrcoef(visits, purchase)[0, 1]
r2 = np.corrcoef(time_spent, purchase)[0, 1]

print("\n3. Correlation Coefficients")
print("Website Visits and Purchase:", round(r1, 4))
print("Time Spent and Purchase:", round(r2, 4))

# 4. Identify the stronger relationship
print("\n4. Analysis")
if abs(r1) > abs(r2):
    print("Website visits have the stronger linear association.")
elif abs(r2) > abs(r1):
    print("Time spent has the stronger linear association.")
else:
    print("Both metrics have equal linear association.")

# 5. Marketing strategies
print("\n5. Marketing Strategies")
print("1. Provide personalized product recommendations.")
print("2. Offer discounts to interested customers.")
print("3. Improve website navigation and engagement.")
print("4. Use customer browsing data for targeted campaigns.")
```

OUTPUT:

```
========== RESTART: C:/Users/Madaraboina Gayathri/OneDrive/Desktop/DSA0402-PODS/ASSESSMENTS/CO3-AT2/Q2.py ==========
E-COMMERCE CUSTOMER BEHAVIOUR ANALYSIS
--------------------------------------

1. Purchase Statistics
Mean: 4970.0
Variance: 8793444.44
Standard Deviation: 2965.37

2. Covariance
Website Visits and Purchase: 22568.89
Time Spent and Purchase: 47266.67

3. Correlation Coefficients
Website Visits and Purchase: 0.9914
Time Spent and Purchase: 0.9949

4. Analysis
Time spent has the stronger linear association.

5. Marketing Strategies
1. Provide personalized product recommendations.
2. Offer discounts to interested customers.
3. Improve website navigation and engagement.
4. Use customer browsing data for targeted campaigns.
```

3.

CODE:

```
import numpy as np
```

```
# Employee dataset
training = np.array([5, 8, 10, 12, 15, 18, 20, 22, 25, 28])

projects = np.array([2, 3, 4, 5, 6, 7, 8, 9, 10, 11])

performance = np.array([60, 65, 70, 74, 79, 84, 88, 91, 95, 98])

print("EMPLOYEE PRODUCTIVITY AND PERFORMANCE")
print("-------------------------------------")

# 1. Mean, variance and standard deviation
print("\n1. Performance Statistics")
print("Mean:", round(np.mean(performance), 2))
print("Variance:", round(np.var(performance, ddof=1), 2))
print("Standard Deviation:", round(np.std(performance, ddof=1), 2))

# 2. Covariance
print("\n2. Covariance")
print("Training Hours and Performance:",
     round(np.cov(training, performance)[0, 1], 2))
print("Projects and Performance:",
     round(np.cov(projects, performance)[0, 1], 2))

# 3. Correlation coefficients
r1 = np.corrcoef(training, performance)[0, 1]
r2 = np.corrcoef(projects, performance)[0, 1]

print("\n3. Correlation Coefficients")
print("Training and Performance:", round(r1, 4))
print("Projects and Performance:", round(r2, 4))

# 4. Identify the strongest factor
print("\n4. Strongest Factor")
if abs(r1) > abs(r2):
   print("Training hours have the stronger association.")
elif abs(r2) > abs(r1):
   print("Projects completed have the stronger association.")
else:
   print("Both factors have equal association.")

# 5. HR recommendations
print("\n5. HR Recommendations")
print("1. Provide regular employee training.")
print("2. Assign meaningful projects.")
print("3. Encourage teamwork and mentoring.")
print("4. Evaluate employee performance regularly.")

print("\nConclusion:")
print("Training and projects are positively associated")
print("with employee performance in this dataset.")
```

OUTPUT:

```
========== RESTART: C:/Users/Mada
EMPLOYEE PRODUCTIVITY AND PERFORMANCE
-------------------------------------

1. Performance Statistics
Mean: 80.4
Variance: 167.82
Standard Deviation: 12.95

2. Covariance
Training Hours and Performance: 97.87
Projects and Performance: 39.11

3. Correlation Coefficients
Training and Performance: 0.9957
Projects and Performance: 0.9972

4. Strongest Factor
Projects completed have the stronger association.

5. HR Recommendations
1. Provide regular employee training.
2. Assign meaningful projects.
3. Encourage teamwork and mentoring.
4. Evaluate employee performance regularly.

Conclusion:
Training and projects are positively associated
with employee performance in this dataset.
```

4.

CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

traffic = [12000, 13500, 14500, 16000, 17500,
           19000, 21000, 22500, 24000, 26000]

rain = [0, 5, 10, 15, 20, 25, 30, 35, 40, 45]

speed = [55, 50, 48, 45, 42, 38, 35, 32, 28, 25]

accidents = [2, 3, 4, 5, 6, 8, 10, 12, 14, 16]

df = pd.DataFrame({
    "Traffic_Volume": traffic,
    "Rainfall": rain,
    "Average_Speed": speed,
    "Accident_Count": accidents
})

print("QUESTION 4: SMART CITY TRAFFIC ANALYSIS")
```

```
print("\n1. Descriptive Statistics")
print("Mean:\n", df.mean())
print("\nVariance:\n", df.var())
print("\nStandard Deviation:\n", df.std())

print("\n2. Covariance Matrix")
print(df.cov().round(2))

print("\n3. Correlation Matrix")
print(df.corr().round(4))

print("\n4. Relationships")
print("Traffic volume and accidents: Positive relationship")
print("Rainfall and accidents: Positive relationship")
print("Average speed and accidents: Negative relationship")

print("\n5. Factor Most Associated with Accidents")
correlations = df.corr()["Accident_Count"].drop("Accident_Count")
print(correlations)
print("Strongest association:",
    correlations.abs().idxmax())

print("\n6. Recommendations")
print("1. Monitor traffic congestion.")
print("2. Improve road drainage during rainfall.")
print("3. Install speed monitoring systems.")
print("4. Increase road safety awareness.")

# 7. Scatter plots
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.scatter(traffic, accidents)
plt.xlabel("Traffic Volume")
plt.ylabel("Accident Count")
plt.title("Traffic vs Accidents")

plt.subplot(1, 3, 2)
plt.scatter(rain, accidents)
plt.xlabel("Rainfall (mm)")
plt.ylabel("Accident Count")
plt.title("Rainfall vs Accidents")

plt.subplot(1, 3, 3)
plt.scatter(speed, accidents)
plt.xlabel("Average Speed (km/h)")
plt.ylabel("Accident Count")
plt.title("Speed vs Accidents")
```

```
plt.tight_layout()
plt.show()

print("\nConclusion:")
print("Accidents increase with traffic volume and rainfall")
print("and are negatively associated with average speed.")
```

OUTPUT:

```
Standard Deviation:
 Traffic_Volume    4706.968829
Rainfall            15.138252
Average_Speed        9.886017
Accident_Count       4.830459
dtype: float64

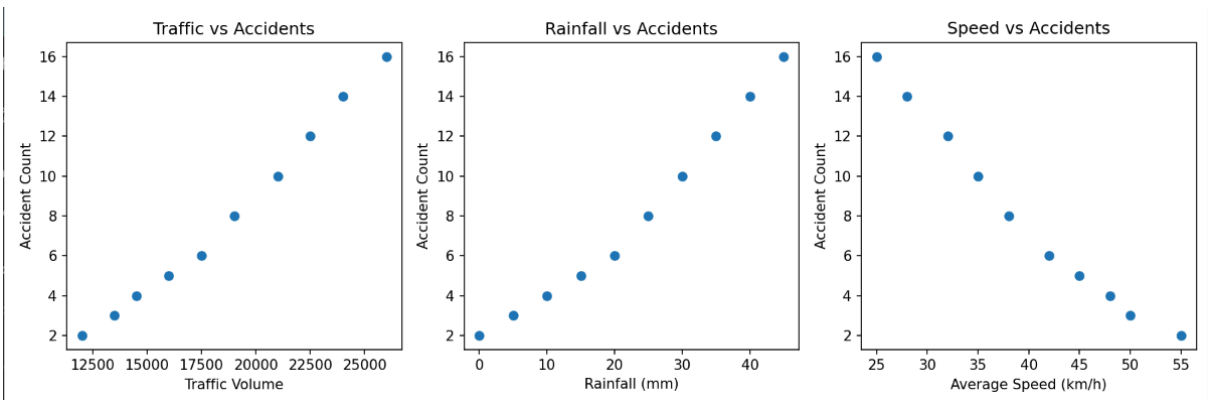
2. Covariance Matrix
                Traffic_Volume  Rainfall  Average_Speed  Accident_Count
Traffic_Volume     22155555.56  71111.11      -46422.22        22611.11
Rainfall              71111.11    229.17        -149.44           72.22
Average_Speed        -46422.22   -149.44          97.73          -47.22
Accident_Count        22611.11     72.22         -47.22           23.33

3. Correlation Matrix
                Traffic_Volume  Rainfall  Average_Speed  Accident_Count
Traffic_Volume          1.0000    0.9980        -0.9976          0.9945
Rainfall                0.9980    1.0000        -0.9986          0.9877
Average_Speed          -0.9976   -0.9986         1.0000         -0.9889
Accident_Count          0.9945    0.9877        -0.9889          1.0000

4. Relationships
Traffic volume and accidents: Positive relationship
Rainfall and accidents: Positive relationship
Average speed and accidents: Negative relationship

5. Factor Most Associated with Accidents
Traffic_Volume    0.994471
Rainfall          0.987658
Average_Speed    -0.988864
Name: Accident_Count, dtype: float64
Strongest association: Traffic_Volume

6. Recommendations
1. Monitor traffic congestion.
2. Improve road drainage during rainfall.
3. Install speed monitoring systems.
4. Increase road safety awareness.
```



5.
CODE:

```
import numpy as np

# Delivery times in minutes
delivery = np.array([
    22, 25, 18, 30, 27, 24, 21, 19,
    26, 23, 28, 20, 24, 25, 22
])

# Calculate point estimates
mean = np.mean(delivery)
variance = np.var(delivery, ddof=1)

print("AVERAGE DELIVERY TIME")
print("---------------------")
print("Delivery times:", delivery)
print("1. Point estimate of population mean:",
      round(mean, 2), "minutes")
print("2. Point estimate of population variance:",
      round(variance, 2))
print("3. Interpretation:")
print("The average delivery time is", round(mean, 2), "minutes.")
print("The variance indicates the variability in delivery times.")
```

OUTPUT:

```
============================== RESTART: C:/Users/Madaraboina Gayathri
AVERAGE DELIVERY TIME
---------------------
Delivery times: [22 25 18 30 27 24 21 19 26 23 28 20 24 25 22]
1. Point estimate of population mean: 23.6 minutes
2. Point estimate of population variance: 11.4
3. Interpretation:
The average delivery time is 23.6 minutes.
The variance indicates the variability in delivery times.
```

6.
CODE:

```
import numpy as np

# Sample customer spending data
spending = np.array([
    120, 145, 130, 155, 160, 125,
    140, 135, 150, 170, 165, 155
])

# Calculate point estimates
mean_spending = np.mean(spending)
std_deviation = np.std(spending, ddof=1)
```

```
print("CUSTOMER SPENDING ANALYSIS")
print("-------------------------")
print("Customer spending:", spending)
print("Number of customers:", len(spending))

print("1. Average spending:",
      round(mean_spending, 2))

print("2. Sample standard deviation:",
      round(std_deviation, 2))

print("3. Why point estimates are useful:")
print("Point estimates summarize population characteristics")
print("using sample data.")
print("The mean estimates average customer spending,")
print("while standard deviation measures variation.")
```

OUTPUT:

```
============================== RESTART: C:/Users/Madaraboina Gayathri/Or
CUSTOMER SPENDING ANALYSIS
-------------------------
Customer spending: [120 145 130 155 160 125 140 135 150 170 165 155]
Number of customers: 12
1. Average spending: 145.83
2. Sample standard deviation: 16.07
3. Why point estimates are useful:
Point estimates summarize population characteristics
using sample data.
The mean estimates average customer spending,
while standard deviation measures variation.
```

7.

CODE:

```
import numpy as np

# Product ratings
ratings = np.array([
    4, 5, 3, 4, 5, 4, 3, 5, 4, 4,
    5, 3, 4, 5, 4, 3, 5, 4, 5, 4
])

# Calculate statistical measures
sample_mean = np.mean(ratings)
sample_variance = np.var(ratings, ddof=1)
sample_std = np.std(ratings, ddof=1)
standard_error = sample_std / np.sqrt(len(ratings))

print("PRODUCT RATINGS ANALYSIS")
print("------------------------")
print("Product ratings:", ratings)
print("Number of ratings:", len(ratings))
```

```
print("1. Sample mean:", round(sample_mean, 2))
print("2. Sample variance:", round(sample_variance, 2))
print("3. Standard error of mean:", round(standard_error, 3))

print("4. Explanation:")
print("Standard error measures the precision of the sample mean.")
print("A smaller standard error indicates a more precise estimate.")
```

OUTPUT:

```
============================== RESTART: C:/Users/Madaraboina Gayathri/
PRODUCT RATINGS ANALYSIS
------------------------
Product ratings: [4 5 3 4 5 4 3 5 4 4 5 3 4 5 4 3 5 4 5 4]
Number of ratings: 20
1. Sample mean: 4.15
2. Sample variance: 0.56
3. Standard error of mean: 0.167
4. Explanation:
Standard error measures the precision of the sample mean.
A smaller standard error indicates a more precise estimate.
```

8.

CODE:

```
import numpy as np

# Exam scores
scores = np.array([
    72, 75, 68, 80, 77, 73, 79, 81, 76, 74,
    78, 69, 70, 82, 71, 75, 77, 73, 80, 76,
    74, 79, 72, 81, 78
])

# Calculate statistical measures
mean_score = np.mean(scores)
std_deviation = np.std(scores, ddof=1)
standard_error = std_deviation / np.sqrt(len(scores))

print("EXAM SCORES ANALYSIS")
print("--------------------")
print("Exam scores:", scores)
print("Number of students:", len(scores))

print("1. Mean score:", round(mean_score, 2))
print("2. Standard deviation:", round(std_deviation, 2))
print("3. Standard error:", round(standard_error, 2))

print("4. Comparison:")
print("Standard deviation measures variation in individual scores.")
print("Standard error measures the precision of the sample mean.")
```

OUTPUT:

```
========== RESTART: C:/Users/Madaraboina Gayathri/OneDrive/Desktop/DS
EXAM SCORES ANALYSIS
--------------------
Exam scores: [72 75 68 80 77 73 79 81 76 74 78 69 70 82 71 75 77 73 80 76 74 79 72 81
 78]
Number of students: 25
1. Mean score: 75.6
2. Standard deviation: 3.96
3. Standard error: 0.79
4. Comparison:
Standard deviation measures variation in individual scores.
Standard error measures the precision of the sample mean.
```

9.

CODE:

```
import numpy as np
from scipy import stats

# Given sample information
sample_size = 50
sample_mean = 498
sample_std = 12
confidence_level = 0.95

# Calculate the 95% confidence interval using t-distribution
alpha = 1 - confidence_level
standard_error = sample_std / np.sqrt(sample_size)
t_value = stats.t.ppf(1 - alpha / 2, df=sample_size - 1)

margin_error = t_value * standard_error
lower_limit = sample_mean - margin_error
upper_limit = sample_mean + margin_error

print("MANUFACTURING WEIGHTS ANALYSIS")
print("------------------------------")
print("Sample size:", sample_size)
print("Sample mean:", sample_mean, "g")
print("Sample standard deviation:", sample_std, "g")
print("1. 95% Confidence Interval:")
print("Lower limit:", round(lower_limit, 2), "g")
print("Upper limit:", round(upper_limit, 2), "g")

print("2. Interpretation:")
print("The interval estimates the population mean weight.")

print("3. Is the target weight of 500 g plausible?")
if lower_limit <= 500 <= upper_limit:
    print("Yes, 500 g lies within the confidence interval.")
else:
    print("No, 500 g lies outside the confidence interval.")
```

OUTPUT:

```
============================== RESTART: C:/Users/Madaraboina Gay
MANUFACTURING WEIGHTS ANALYSIS
------------------------------
Sample size: 50
Sample mean: 498 g
Sample standard deviation: 12 g
1. 95% Confidence Interval:
Lower limit: 494.59 g
Upper limit: 501.41 g
2. Interpretation:
The interval estimates the population mean weight.
3. Is the target weight of 500 g plausible?
Yes, 500 g lies within the confidence interval.
```

10.

CODE:

```
# Customer satisfaction data
mean_A = 4.3
lower_A = 4.1
upper_A = 4.5

mean_B = 4.0
lower_B = 3.8
upper_B = 4.2

print("CUSTOMER SATISFACTION ANALYSIS")
print("-----------------------------")

print("Branch A mean:", mean_A)
print("Branch A 95% Confidence Interval:",
     (lower_A, upper_A))

print("Branch B mean:", mean_B)
print("Branch B 95% Confidence Interval:",
     (lower_B, upper_B))

print("\n1. Comparison of mean satisfaction:")
if mean_A > mean_B:
   print("Branch A has a higher mean satisfaction score.")
else:
   print("Branch B has a higher mean satisfaction score.")

print("\n2. Is Branch A better?")
if lower_A > upper_B:
   print("Yes, Branch A has a clearly higher confidence interval.")
else:
   print("The confidence intervals overlap, so the difference")
   print("is not conclusive from these intervals alone.")

print("\n3. Conclusion:")
print("Branch A has a higher average score, but further")
```

print("statistical testing may be needed to confirm the difference.")

OUTPUT:

```
========== RESTART: C:/Users/Madaraboina Gayathr
CUSTOMER SATISFACTION ANALYSIS
------------------------------
Branch A mean: 4.3
Branch A 95% Confidence Interval: (4.1, 4.5)
Branch B mean: 4.0
Branch B 95% Confidence Interval: (3.8, 4.2)

1. Comparison of mean satisfaction:
Branch A has a higher mean satisfaction score.

2. Is Branch A better?
The confidence intervals overlap, so the difference
is not conclusive from these intervals alone.

3. Conclusion:
Branch A has a higher average score, but further
statistical testing may be needed to confirm the difference.
```

11.
CODE:

```
import numpy as np
from scipy import stats

# Given sample information
claimed_mean = 10
sample_size = 36
sample_mean = 9.5
sample_std = 1.8
alpha = 0.05

# Hypothesis testing
H0 = "Population mean battery life = 10 hours"
H1 = "Population mean battery life != 10 hours"

# Calculate t-statistic and p-value
standard_error = sample_std / np.sqrt(sample_size)
t_statistic = (sample_mean - claimed_mean) / standard_error
degrees_freedom = sample_size - 1
p_value = 2 * stats.t.cdf(t_statistic, df=degrees_freedom)

print("AVERAGE BATTERY LIFE ANALYSIS")
print("-----------------------------")
print("Null hypothesis (H0):", H0)
print("Alternative hypothesis (H1):", H1)
print("Sample size:", sample_size)
print("Sample mean:", sample_mean, "hours")
print("Sample standard deviation:", sample_std, "hours")
print("Significance level:", alpha)
```

```
print("\nT-statistic:", round(t_statistic, 3))
print("P-value:", round(p_value, 4))

if p_value < alpha:
    print("\nDecision: Reject the null hypothesis.")
    print("Conclusion: The mean battery life differs significantly")
    print("from 10 hours.")
else:
    print("\nDecision: Fail to reject the null hypothesis.")
    print("Conclusion: There is insufficient evidence that")
    print("the mean battery life differs from 10 hours.")
```

OUTPUT:

```
========== RESTART: C:/Users/Madaraboina Gayathri/Onel
AVERAGE BATTERY LIFE ANALYSIS
-----------------------------
Null hypothesis (H0): Population mean battery life = 10 hours
Alternative hypothesis (H1): Population mean battery life != 10 hours
Sample size: 36
Sample mean: 9.5 hours
Sample standard deviation: 1.8 hours
Significance level: 0.05

T-statistic: -1.667
P-value: 0.1045

Decision: Fail to reject the null hypothesis.
Conclusion: There is insufficient evidence that
the mean battery life differs from 10 hours.
```

12.
CODE:

```
import numpy as np
from scipy import stats

# Sales before and after the marketing campaign
before = np.array([50, 52, 49, 55, 53, 51, 54, 50])
after = np.array([58, 60, 57, 62, 59, 61, 63, 58])

alpha = 0.05

# Calculate differences
differences = after - before

# Hypothesis testing using a paired t-test
t_statistic, p_value_two_tailed = stats.ttest_rel(after, before)

# One-tailed p-value: test whether sales increased
if t_statistic >= 0:
    p_value = p_value_two_tailed / 2
```

```
else:
    p_value = 1 - p_value_two_tailed / 2

print("MARKETING CAMPAIGN ANALYSIS")
print("---------------------------")
print("Sales before:", before)
print("Sales after:", after)
print("Average sales before:", round(np.mean(before), 2))
print("Average sales after:", round(np.mean(after), 2))
print("Average increase:", round(np.mean(differences), 2))

print("\nH0: The campaign does not increase average sales.")
print("H1: The campaign increases average sales.")
print("T-statistic:", round(t_statistic, 3))
print("One-tailed p-value:", round(p_value, 6))

if p_value < alpha:
    print("\nDecision: Reject H0.")
    print("Conclusion: The campaign significantly increases sales.")
else:
    print("\nDecision: Fail to reject H0.")
    print("Conclusion: There is insufficient evidence of increased sales.")
```

OUTPUT:

```
= RESTART: C:/Users/Madaraboina Gayathri/OneDrive/Desktop/DSA
MARKETING CAMPAIGN ANALYSIS
---------------------------
Sales before: [50 52 49 55 53 51 54 50]
Sales after: [58 60 57 62 59 61 63 58]
Average sales before: 51.75
Average sales after: 59.75
Average increase: 8.0

H0: The campaign does not increase average sales.
H1: The campaign increases average sales.
T-statistic: 18.931
One-tailed p-value: 0.0

Decision: Reject H0.
Conclusion: The campaign significantly increases sales.
```

13.

CODE:

```
import numpy as np
from scipy import stats

# Given sample information
sample_size = 100
sample_mean = 4.8
sample_std = 1.5
confidence_level = 0.95
```

```
# Calculate standard error
standard_error = sample_std / np.sqrt(sample_size)

# Calculate 95% confidence interval using t-distribution
alpha = 1 - confidence_level
t_value = stats.t.ppf(1 - alpha / 2, df=sample_size - 1)

margin_error = t_value * standard_error
lower_limit = sample_mean - margin_error
upper_limit = sample_mean + margin_error

print("CALL CENTRE RESPONSE TIME ANALYSIS")
print("----------------------------------")
print("Sample size:", sample_size)
print("Sample mean:", sample_mean, "minutes")
print("Sample standard deviation:", sample_std, "minutes")

print("\n1. Point estimate of population mean:")
print(round(sample_mean, 2), "minutes")

print("\n2. Standard error:")
print(round(standard_error, 3), "minutes")

print("\n3. 95% Confidence Interval:")
print("Lower limit:", round(lower_limit, 3), "minutes")
print("Upper limit:", round(upper_limit, 3), "minutes")

print("\n4. Frequentist interpretation:")
print("The method produces intervals that contain the true")
print("population mean in approximately 95% of repeated samples.")
```

OUTPUT:

```
========== RESTART: C:/Users/Madaraboina Gayath
CALL CENTRE RESPONSE TIME ANALYSIS
----------------------------------
Sample size: 100
Sample mean: 4.8 minutes
Sample standard deviation: 1.5 minutes

1. Point estimate of population mean:
4.8 minutes

2. Standard error:
0.15 minutes

3. 95% Confidence Interval:
Lower limit: 4.502 minutes
Upper limit: 5.098 minutes

4. Frequentist interpretation:
The method produces intervals that contain the true
population mean in approximately 95% of repeated samples.
```

14.
CODE:

```
import numpy as np
from scipy import stats

# Given sample information
sample_size = 500
defective_products = 22
confidence_level = 0.95

# Calculate the sample defect rate
defect_rate = defective_products / sample_size

# Calculate standard error for the proportion
standard_error = np.sqrt(
    defect_rate * (1 - defect_rate) / sample_size
)

# Calculate 95% confidence interval using normal approximation
z_value = stats.norm.ppf(1 - (1 - confidence_level) / 2)

margin_error = z_value * standard_error
lower_limit = defect_rate - margin_error
upper_limit = defect_rate + margin_error

print("DEFECTIVE PRODUCTS ANALYSIS")
print("---------------------------")
print("Sample size:", sample_size)
print("Defective products:", defective_products)

print("\n1. Estimated defect rate:")
```

```
print(round(defect_rate * 100, 2), "%")

print("\n2. Standard error:")
print(round(standard_error, 5))

print("\n3. 95% Confidence Interval:")
print("Lower limit:", round(lower_limit * 100, 2), "%")
print("Upper limit:", round(upper_limit * 100, 2), "%")

print("\n4. Frequentist interpretation:")
print("In repeated sampling, approximately 95% of intervals")
print("constructed by this method would contain the true")
print("population defect rate.")
```

OUTPUT:

```
========== RESTART: C:/Users/Madaraboina
DEFECTIVE PRODUCTS ANALYSIS
---------------------------
Sample size: 500
Defective products: 22

1. Estimated defect rate:
4.4 %

2. Standard error:
0.00917

3. 95% Confidence Interval:
Lower limit: 2.6 %
Upper limit: 6.2 %

4. Frequentist interpretation:
In repeated sampling, approximately 95% of intervals
constructed by this method would contain the true
population defect rate.
```

15.

CODE:

```
# Given probabilities
P_spam = 0.25
P_not_spam = 1 - P_spam

P_offer_given_spam = 0.80
P_offer_given_not_spam = 0.10

# Calculate total probability of an email containing "Offer"
P_offer = (
    P_offer_given_spam * P_spam
    + P_offer_given_not_spam * P_not_spam
)

# Apply Bayes' theorem
```

```
P_spam_given_offer = (
   P_offer_given_spam * P_spam
) / P_offer

print("EMAIL SPAM DETECTION")
print("--------------------")
print("P(Spam):", P_spam)
print("P(Not Spam):", round(P_not_spam, 2))
print("P(Offer | Spam):", P_offer_given_spam)
print("P(Offer | Not Spam):", P_offer_given_not_spam)
print("P(Offer):", round(P_offer, 3))

print("\nProbability that an email is spam given 'Offer':")
print(round(P_spam_given_offer, 4))

print("Spam probability in percentage:",
     round(P_spam_given_offer * 100, 2), "%")

print("\nInterpretation:")
print("An email containing 'Offer' has a",
     round(P_spam_given_offer * 100, 2),
     "% probability of being spam.")
```

OUTPUT:

```
============================== RESTART: C:/Users/Madaraboina Gayathri/O
EMAIL SPAM DETECTION
--------------------
P(Spam): 0.25
P(Not Spam): 0.75
P(Offer | Spam): 0.8
P(Offer | Not Spam): 0.1
P(Offer): 0.275

Probability that an email is spam given 'Offer':
0.7273
Spam probability in percentage: 72.73 %

Interpretation:
An email containing 'Offer' has a 72.73 % probability of being spam.
```

16.

CODE:

```
# Given probabilities
P_disease = 0.02
P_no_disease = 1 - P_disease

P_positive_given_disease = 0.95
P_negative_given_no_disease = 0.90

# Probability of a positive result for a person without disease
P_positive_given_no_disease = 1 - P_negative_given_no_disease
```

```
# Total probability of a positive test
P_positive = (
    P_positive_given_disease * P_disease
    + P_positive_given_no_disease * P_no_disease
)

# Bayes' theorem
P_disease_given_positive = (
    P_positive_given_disease * P_disease
) / P_positive

print("DISEASE DIAGNOSIS ANALYSIS")
print("-------------------------")
print("Disease prevalence:", P_disease)
print("Test sensitivity:", P_positive_given_disease)
print("Test specificity:", P_negative_given_no_disease)

print("\nProbability of a positive test:",
      round(P_positive, 4))

print("Probability of disease given a positive test:",
      round(P_disease_given_positive, 4))

print("Posterior probability in percentage:",
      round(P_disease_given_positive * 100, 2), "%")

print("\nInterpretation:")
print("A positive test does not guarantee that a person")
print("has the disease. The result also depends on disease prevalence.")
```

OUTPUT:

```
============================== RESTART: C:/Users/Madaraboina Gayathr
DISEASE DIAGNOSIS ANALYSIS
-------------------------
Disease prevalence: 0.02
Test sensitivity: 0.95
Test specificity: 0.9

Probability of a positive test: 0.117
Probability of disease given a positive test: 0.1624
Posterior probability in percentage: 16.24 %

Interpretation:
A positive test does not guarantee that a person
has the disease. The result also depends on disease prevalence.
```

17.

CODE:

```
import numpy as np
from scipy import stats
```

```
# Website data
visitors_A = 5000
purchases_A = 250

visitors_B = 5200
purchases_B = 312

alpha = 0.05

# Calculate conversion rates
rate_A = purchases_A / visitors_A
rate_B = purchases_B / visitors_B

# Hypotheses
# H0: pB = pA
# H1: pB > pA

# Pooled proportion under the null hypothesis
pooled_rate = (
    purchases_A + purchases_B
) / (visitors_A + visitors_B)

# Standard error under the null hypothesis
standard_error = np.sqrt(
    pooled_rate * (1 - pooled_rate)
    * (1 / visitors_A + 1 / visitors_B)
)

# Z-statistic
z_statistic = (rate_B - rate_A) / standard_error

# One-tailed p-value
p_value = stats.norm.sf(z_statistic)

print("WEBSITE CONVERSION RATE ANALYSIS")
print("--------------------------------")
print("Website A conversion rate:",
      round(rate_A * 100, 2), "%")
print("Website B conversion rate:",
      round(rate_B * 100, 2), "%")

print("\nH0: Website B conversion rate equals Website A.")
print("H1: Website B conversion rate is higher.")

print("Z-statistic:", round(z_statistic, 3))
print("One-tailed p-value:", round(p_value, 5))

if p_value < alpha:
    print("\nDecision: Reject H0.")
```

```
    print("Conclusion: Website B has a significantly higher")
    print("conversion rate at the 5% significance level.")
else:
    print("\nDecision: Fail to reject H0.")
    print("Conclusion: There is insufficient evidence that")
    print("Website B has a higher conversion rate.")
```

OUTPUT:

```
========== RESTART: C:/Users/Madaraboina
WEBSITE CONVERSION RATE ANALYSIS
--------------------------------
Website A conversion rate: 5.0 %
Website B conversion rate: 6.0 %

H0: Website B conversion rate equals Website A.
H1: Website B conversion rate is higher.
Z-statistic: 2.213
One-tailed p-value: 0.01346

Decision: Reject H0.
Conclusion: Website B has a significantly higher
conversion rate at the 5% significance level.
```

18.
CODE:

```
import numpy as np
from scipy import stats

# Given sample data
n_A = 400
mean_A = 18
std_A = 5

n_B = 420
mean_B = 20
std_B = 6

alpha = 0.05

# Hypotheses
# H0: Mean engagement of B = Mean engagement of A
# H1: Mean engagement of B > Mean engagement of A

# Welch's two-sample t-test calculated from summary statistics
standard_error = np.sqrt(
    (std_A ** 2 / n_A) + (std_B ** 2 / n_B)
)

t_statistic = (mean_B - mean_A) / standard_error
```

```
# Welch-Satterthwaite degrees of freedom
numerator = (
   (std_A ** 2 / n_A + std_B ** 2 / n_B) ** 2
)

denominator = (
   (std_A ** 2 / n_A) ** 2 / (n_A - 1)
   + (std_B ** 2 / n_B) ** 2 / (n_B - 1)
)

df = numerator / denominator

# One-tailed p-value
p_value = stats.t.sf(t_statistic, df)

print("MOBILE APP FEATURE TEST")
print("-----------------------")
print("Feature A mean engagement:", mean_A, "minutes")
print("Feature B mean engagement:", mean_B, "minutes")
print("Feature A sample size:", n_A)
print("Feature B sample size:", n_B)

print("\nH0: Feature B does not increase mean engagement.")
print("H1: Feature B increases mean engagement.")

print("T-statistic:", round(t_statistic, 3))
print("Degrees of freedom:", round(df, 2))
print("One-tailed p-value:", round(p_value, 5))

if p_value < alpha:
   print("\nDecision: Reject H0.")
   print("Conclusion: Feature B significantly increases")
   print("average user engagement at the 5% level.")
else:
   print("\nDecision: Fail to reject H0.")
   print("Conclusion: There is insufficient evidence that")
   print("Feature B increases average user engagement.")
```

OUTPUT:

```
========== RESTART: C:/Users/Madaraboina
MOBILE APP FEATURE TEST
-----------------------
Feature A mean engagement: 18 minutes
Feature B mean engagement: 20 minutes
Feature A sample size: 400
Feature B sample size: 420

H0: Feature B does not increase mean engagement.
H1: Feature B increases mean engagement.
T-statistic: 5.195
Degrees of freedom: 803.95
One-tailed p-value: 0.0

Decision: Reject H0.
Conclusion: Feature B significantly increases
average user engagement at the 5% level.
```

20.
CODE:

```
import numpy as np
from scipy import stats

# Given data from 50 test folds
mean_difference = 200
std_difference = 350
n = 50
alpha = 0.05

# Hypotheses
# H0: Mean difference in MAE = 0
# H1: Model B has lower MAE than Model A

# Calculate standard error and t-statistic
standard_error = std_difference / np.sqrt(n)
t_statistic = mean_difference / standard_error

# Degrees of freedom
df = n - 1

# One-tailed p-value
p_value = stats.t.sf(t_statistic, df)

print("SALARY PREDICTION MODEL ANALYSIS")
print("--------------------------------")
print("Model A MAE: 4200")
print("Model B MAE: 4000")
print("Mean difference (A - B):", mean_difference)
print("Standard deviation of differences:", std_difference)
print("Number of test folds:", n)

print("\nH0: Mean difference in MAE is zero.")
```

```
print("H1: Model B has significantly lower MAE.")

print("Standard error:", round(standard_error, 2))
print("T-statistic:", round(t_statistic, 3))
print("Degrees of freedom:", df)
print("One-tailed p-value:", round(p_value, 5))

if p_value < alpha:
    print("\nDecision: Reject H0.")
    print("Conclusion: Model B significantly improves")
    print("salary prediction accuracy.")
else:
    print("\nDecision: Fail to reject H0.")
    print("Conclusion: There is insufficient evidence that")
    print("Model B significantly improves prediction accuracy.")
```

OUTPUT:

```
========== RESTART: C:/Users/M
SALARY PREDICTION MODEL ANALYSIS
--------------------------------
Model A MAE: 4200
Model B MAE: 4000
Mean difference (A - B): 200
Standard deviation of differences: 350
Number of test folds: 50

H0: Mean difference in MAE is zero.
H1: Model B has significantly lower MAE.
Standard error: 49.5
T-statistic: 4.041
Degrees of freedom: 49
One-tailed p-value: 9e-05

Decision: Reject H0.
Conclusion: Model B significantly improves
salary prediction accuracy.
```